Python - Day 7 (project)

Beginner Python project : Hangman

Goal:-

By the end of Day 7, you'll understand how to break down a problem into steps and build a working Hangman game in Python. This Consolidates your knowledge of loops, Conditionals, lists, strings and basic game logic.

Core Concepts Covered

(1). looping logic
- Using while loops to keep the game running until a termind condition (win or lose) is reached
- Using for loops to iterate through letters in a word.

(2) Conditional Statements
- if, elif, and else statment to:
- check if guessed letters are in the secret word
- Decrease lives when a guess is incorrect
- check win/lose conditions

(3) lists and strings
- Representing the secret word as a list of letters
- Displaying progress by turning a list of underscore placeholders into a string.

(4) Random Module
- Choosing a random word from a predefined list for the player to guess

Step-by-step Hangman Game logic
Step 1: Import Modules and Set Up words

```
import random
word_list = ["aardvark", "baboon", "camel",
             "donkey", "elephant"]
# list of possible words
```

- The game picks a secret word randomly using random.choice() so every playthrough can be different.

Step 2: Initialize Game State

```
chosen_word = random.choice(word_list)
display = []
# Create underscore for each letter
for letter in chosen_word:
    display.append("_")
```

- display starts a list of _, one for each character in the secret word.

Step 3: Game loop begins

```
end_of_game = False
lives = 6
```

- end_of_game tracks whether the game should continue
- lives sets how many wrong guesses the player can make before losing.

Step 4: Ask for User input

```
while not end_of_game:
    guess = input("Guess a letter: ").lower()
```

- The loop repeatedly asks the player to guess a letter
- .lower() ensures case-insensitive matching

Step 5: Reveal correct letters

```
for position in range(len(chosen_word)):
    letter = chosen_word[position]
    if letter == guess:
        display[position] = letter
```

- for each letter in the chosen word check if it matches the guessed letter
- Replace underscores in the display list with correct guessed letter.

Step 6: Handle incorrect Guesses

```
if guess not in chosen_word:
    lives -= 1
    print(f"Number of lives: {lives}")
    if lives == 0:
        end_of_game = True
        print("You lose")
```

- if the guess is wrong, the player loses a life
- when lives == 0, the game ends in a loss.

Step 7: win condition

```
print(f"{' '.join(display)}")
if "_" not in display:
    end_of_game = True
    print("You win")
```

- if there are no underscores left (i.e. the word is fully revealed), the player wins.

Step 8: Optional feedback (ASCII Art)

- while not mandatory, students are encouraged to print visual Hangman states (stages) each round to make it fun
- These are often imported from a separate file like hangman_art.py